```
import torch
from torch import nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 21.3MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 520kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.82MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 7.49MB/s]
```

```
def loss_function(recon_x, x, mu, logvar):
    BCE = F.binary_cross_entropy(recon_x, x.view(-1, 784), reduction='sum')
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return BCE + KLD
```

```
class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
        super(VAE, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc2_logvar = nn.Linear(hidden_dim, latent_dim)

        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        h1 = F.relu(self.fc1(x))
        return self.fc2_mu(h1), self.fc2_logvar(h1)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h3 = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h3))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 784))
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar
```

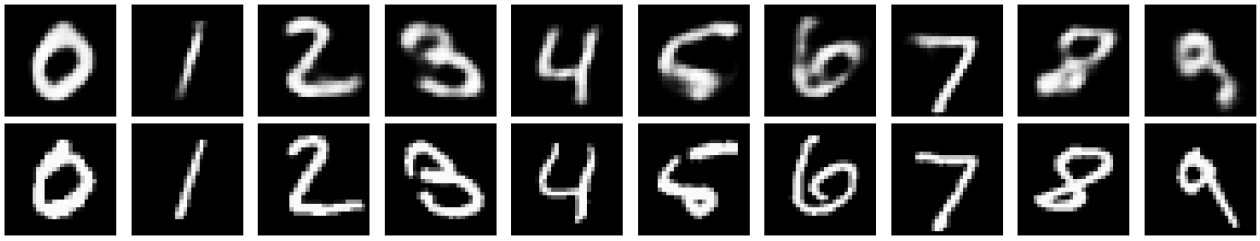
```
def loss_function(recon_x, x, mu, logvar):
    BCE = F.binary_cross_entropy(recon_x, x.view(-1, 784), reduction='sum')
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return BCE + KLD
```

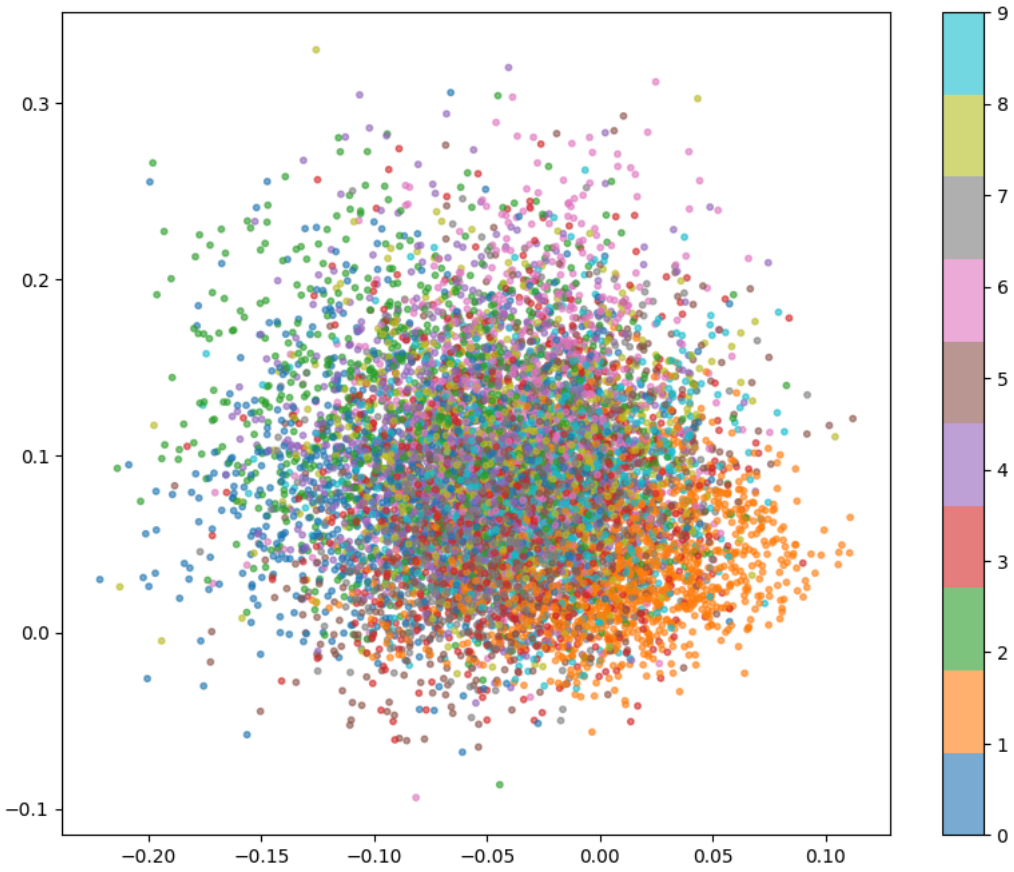
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = VAE().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
epochs = 10
train_losses = []

for epoch in range(epochs):
    model.train()
    train_loss = 0
    for batch_idx, (data, _) in enumerate(train_loader):
        data = data.to(device)
        optimizer.zero_grad()

        recon_batch, mu, logvar = model(data)
        loss = loss_function(recon_batch, data, mu, logvar)

        loss.backward()
        train_loss += loss.item()
        optimizer.step()

    epoch_loss = train_loss / len(train_loader.dataset)
```

```
    train_losses.append(epoch_loss)
    print(f'Epoch {epoch+1}, Loss: {epoch_loss:.4f}')
```

```
Epoch 1, Loss: 164.2163
Epoch 2, Loss: 121.2681
Epoch 3, Loss: 114.4705
Epoch 4, Loss: 111.6074
Epoch 5, Loss: 109.9508
Epoch 6, Loss: 108.8230
Epoch 7, Loss: 108.0029
Epoch 8, Loss: 107.3938
Epoch 9, Loss: 106.8963
Epoch 10, Loss: 106.4882
```

```
model.eval()
real_images = []
labels_found = set()

for data, target in test_loader:
    for i in range(len(target)):
        label = target[i].item()
        if label not in labels_found:
            labels_found.add(label)
            real_images.append((label, data[i]))
        if len(labels_found) == 10:
            break
    if len(labels_found) == 10:
        break

real_images.sort(key=lambda x: x[0])
images_tensor = torch.stack([x[1] for x in real_images]).to(device)

with torch.no_grad():
    reconstructed, _, _ = model(images_tensor)

images_tensor = images_tensor.cpu().view(-1, 28, 28)
reconstructed = reconstructed.cpu().view(-1, 28, 28)

fig, axes = plt.subplots(2, 10, figsize=(15, 3))

for i in range(10):
    axes[0, i].imshow(reconstructed[i], cmap='gray')
    axes[0, i].axis('off')

    axes[1, i].imshow(images_tensor[i], cmap='gray')
    axes[1, i].axis('off')

plt.tight_layout()
plt.show()
```



```
model_2d = VAE(latent_dim=2).to(device)

model_2d.eval()
latents = []
labels = []

with torch.no_grad():
    for data, label in test_loader:
        data = data.to(device)
        mu, _ = model_2d.encode(data.view(-1, 784))
        latents.append(mu.cpu())
        labels.append(label)

latents = torch.cat(latents).numpy()
labels = torch.cat(labels).numpy()

plt.figure(figsize=(10, 8))
```

```
scatter = plt.scatter(latents[:, 0], latents[:, 1], c=labels, cmap='tab10', alpha=0.6, s=10)
plt.colorbar(scatter)
plt.show()
```